

Design and Performance Analysis of a Deterministic Network Stack for Virtualised Protection and Control Systems

Radu-Vladimir Raceala

Advisors: Peter Okelmann, Dr. Anatole Lefort

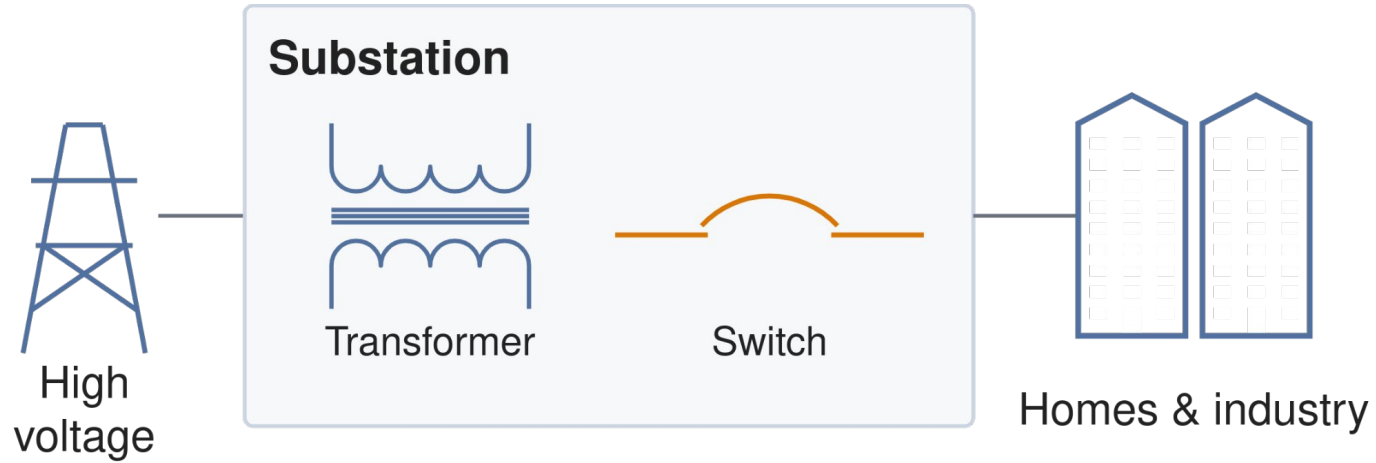
Chair of Distributed Systems and Operating Systems

<https://dse.in.tum.de/>



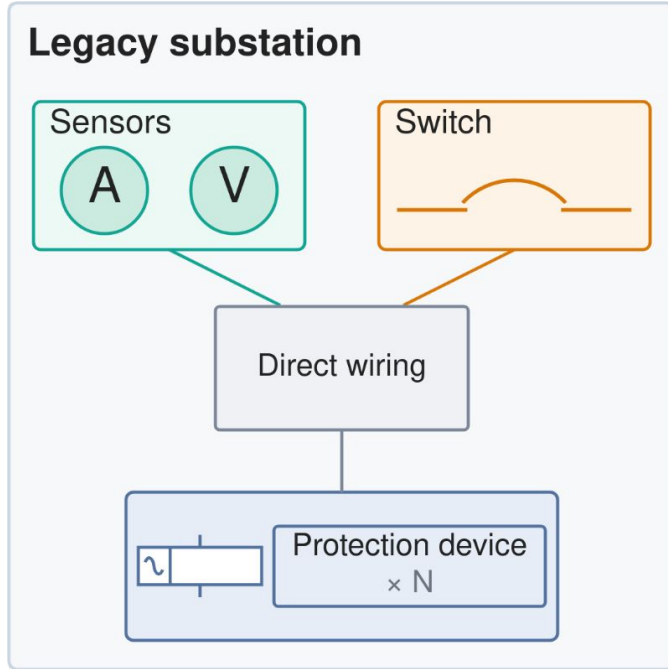
06.05.2026 – 07.09.2026

Context: Substations

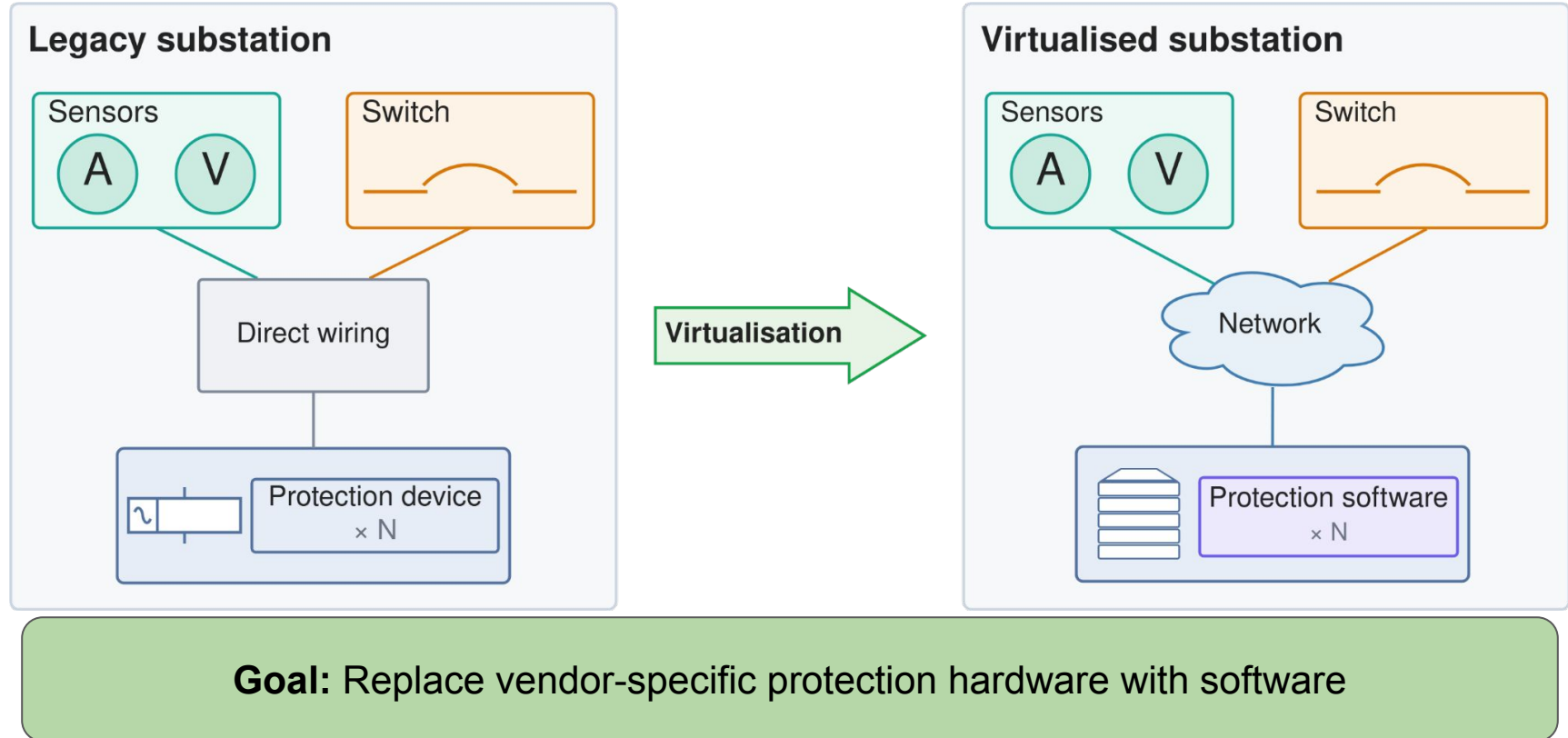


Substations transform voltage and disconnect faulty circuits

Context: Substation Protection & Control



Context: Substation Protection & Control



- Containerised VPACs without network stack exploration
 - Schönborn et al. [1] identify virtual network delays, not scheduling, as the cause of timing failures but do not explore kernel bypassing
- Industry reports validate deployments, not designs
 - ABB [2] and Intel [3] white papers
- Kernel bypassing is proven in other domains
 - E.g, Low-Latency Network Functions, Scientific Computing, Industrial Automation

Previous approaches do not explore **kernel bypassing**

Can software meet the protection deadline of 5 ms, and for how many streams?

Kernel Bypassing in Networking

Packets skip the network stack: the application talks to the NIC.



XDP / AF_XDP

- Runs in the driver, before sk_buff allocation
 - Keeps kernel tooling



DPDK

- Poll-mode driver in user space
 - Complete control, no kernel tooling

Lower, steadier latency and control of packet processing

Low-Latency — trip inside the protection deadline

Determinism — a bounded tail, not just a good mean

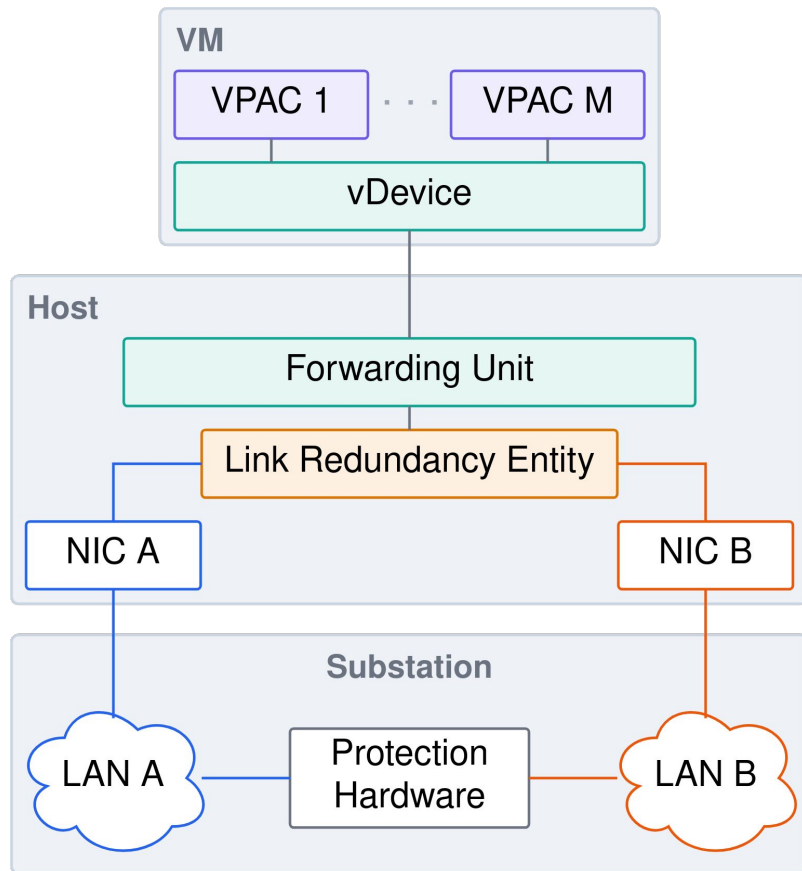
Interoperability — run VPACs unchanged

Resource usage — keeping compute costs low

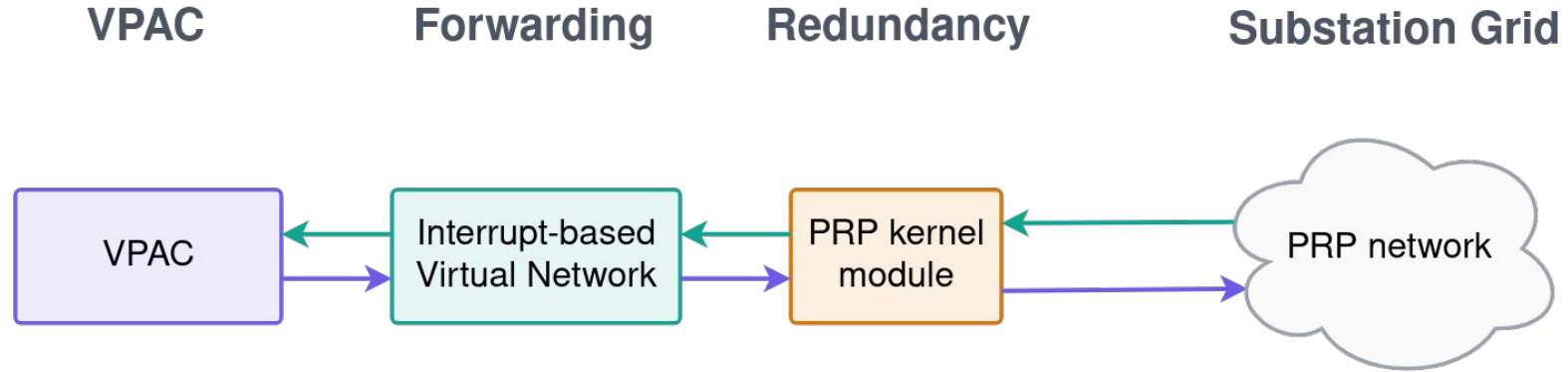
The four pull against each other => **build multiple stacks**

Overview

- VPAC
- vDevice
 - IVSHMEM
 - Virtio-net
- Forwarding Unit
- Link Redundancy Entity
 - The Parallel Redundancy Protocol (PRP)
- Substation Network
 - Close to zero packet loss
 - Bounded latency

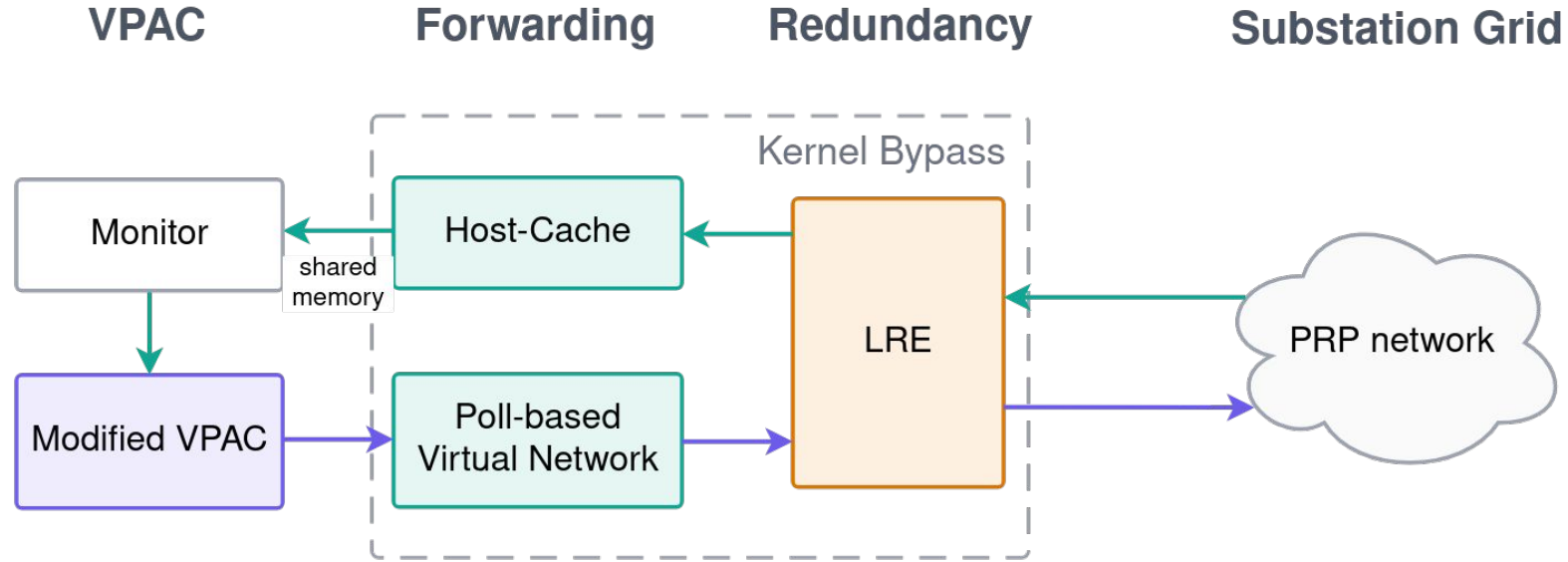


Shared-Nothing Variant



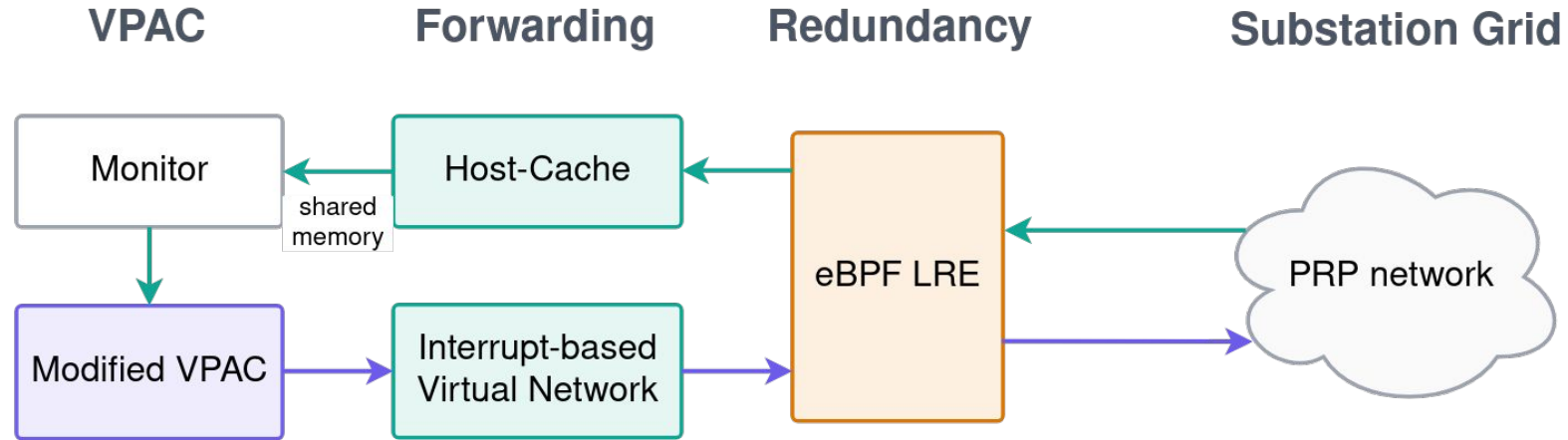
Unmodified VPACs, protection and preconditioning in the VM

DPDK Host-Cache Variant



Modified VPACs, lowest latency, no kernel tooling

AF_XDP Host-Cache Variant



Modified VPACs, preserves kernel tooling

- **IPC protocol**
 - Eventfd-based
 - Costs approximately 3.3 μ s per stream
- **PRP deduplication**
 - Similar to the algorithm used by the kernel module
 - Constant memory and compute

The modified VPACs and the monitor share a core

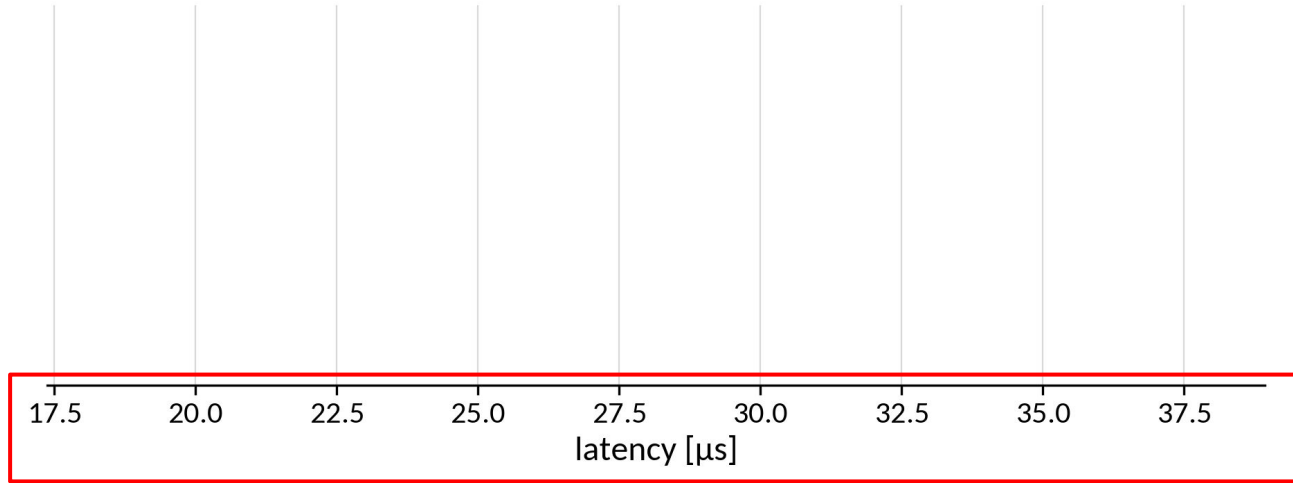
- **Test bench:**

- Two machines, each Intel Xeon Gold 5317 (12 physical cores, 3.00 GHz)
- Intel X550 dual-port 10 GbE, connected back-to-back
- NixOS host, Ubuntu 24.04.4 LTS guest
- Low-latency configuration

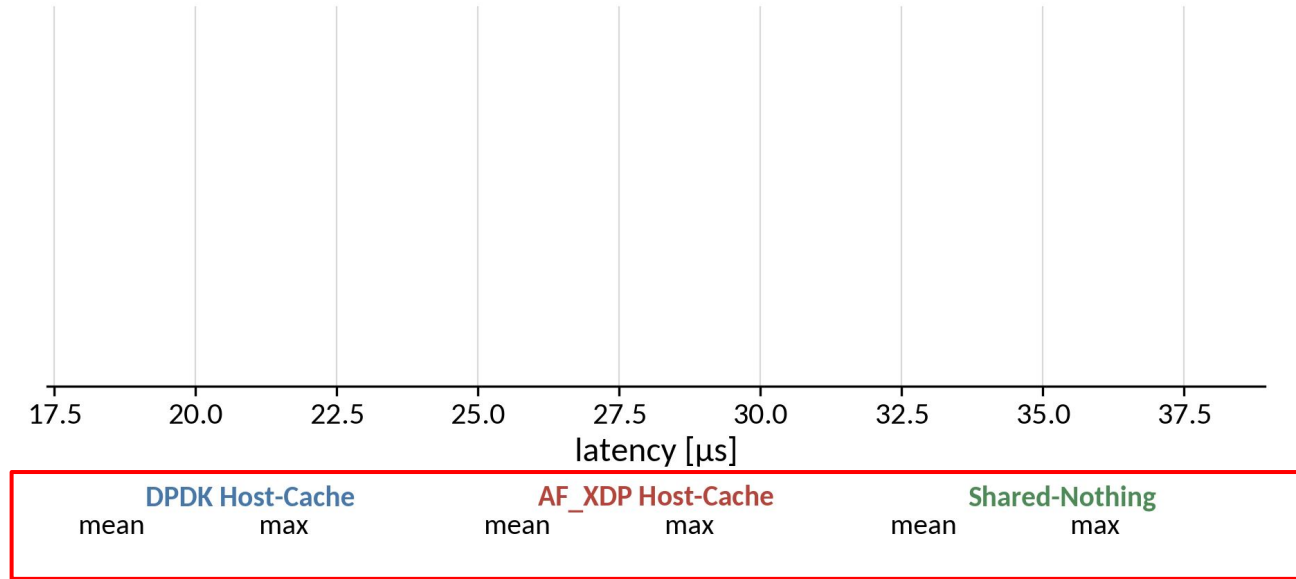
- **Workload:**

- DPDK load generator
- One trip per window of sensor values
- Preconditioning and protection are NOPs

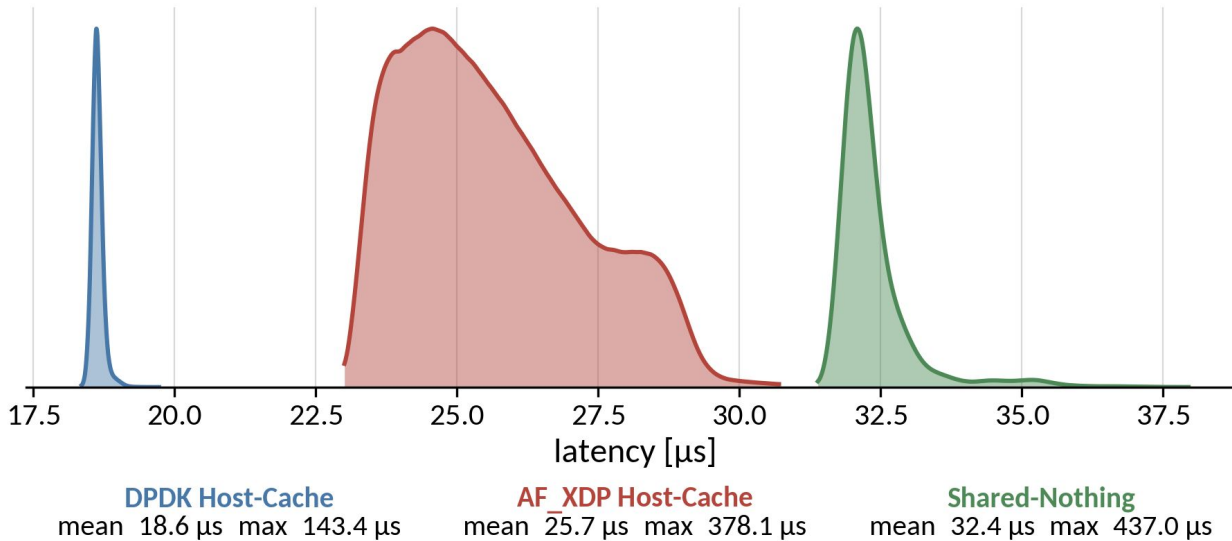
End-to-End Latency



End-to-End Latency

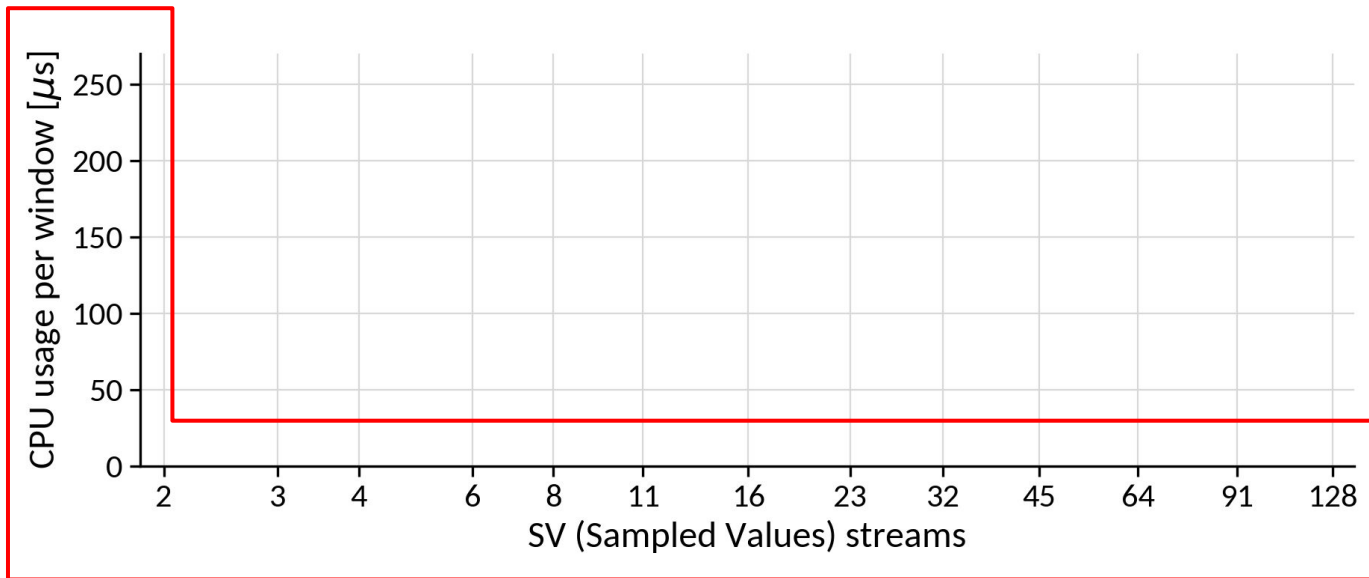


End-to-End Latency

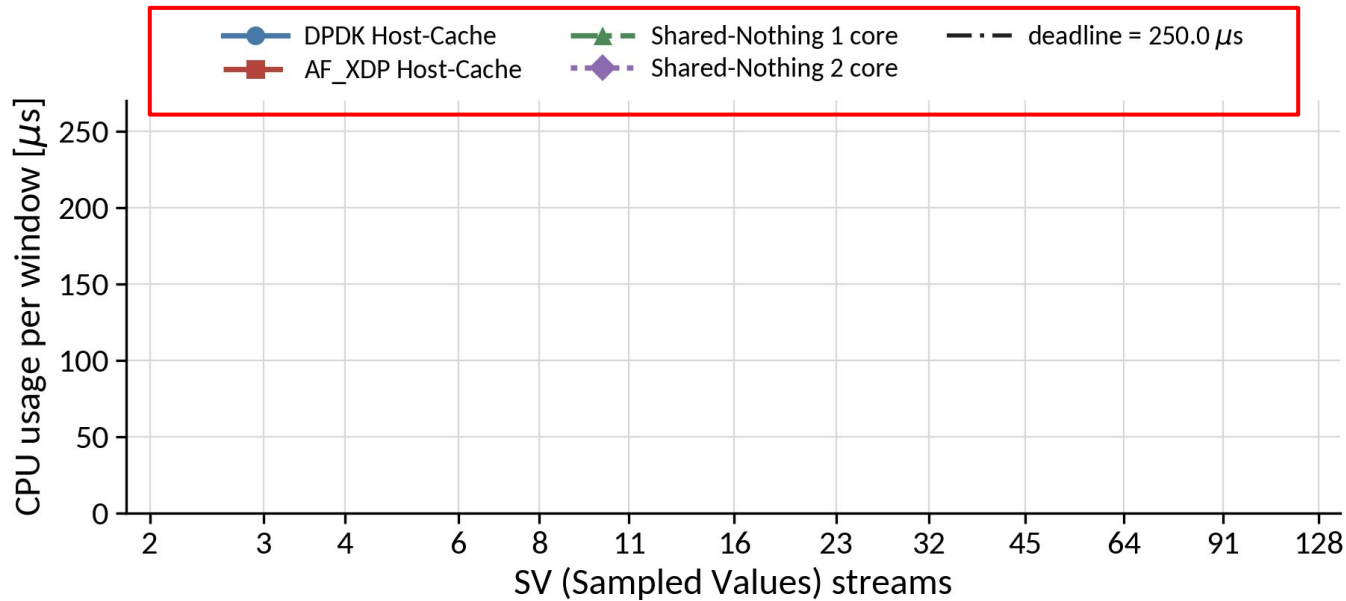


DPDK's maximum observed latency over 40 minutes is 3× lower than Shared-Nothing's

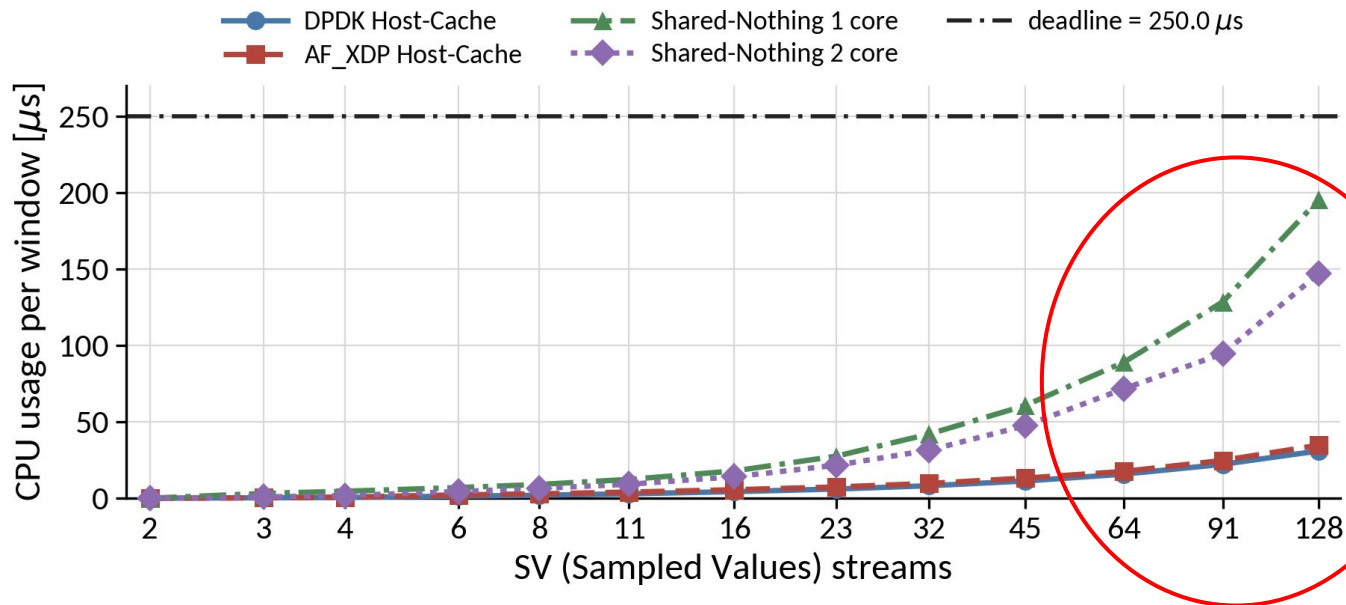
Host-side Capacity



Host-side Capacity

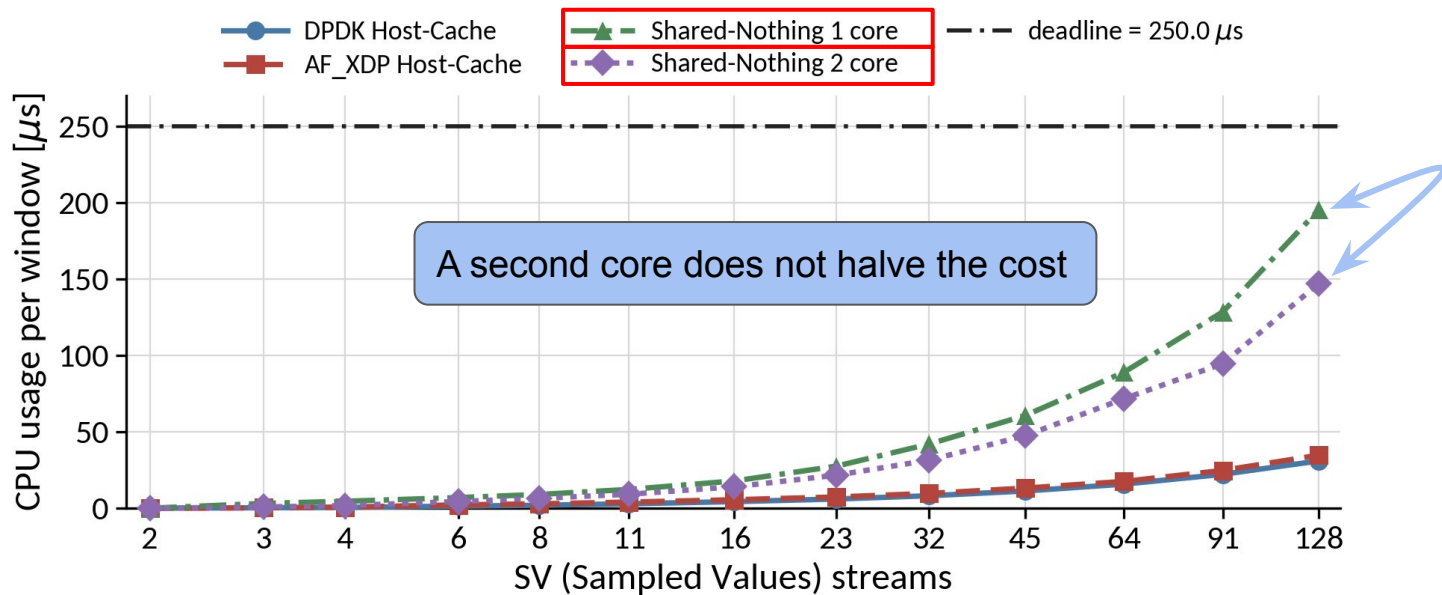


Host-side Capacity



Bypassing the kernel reduces costs at high stream counts

Host-side Capacity



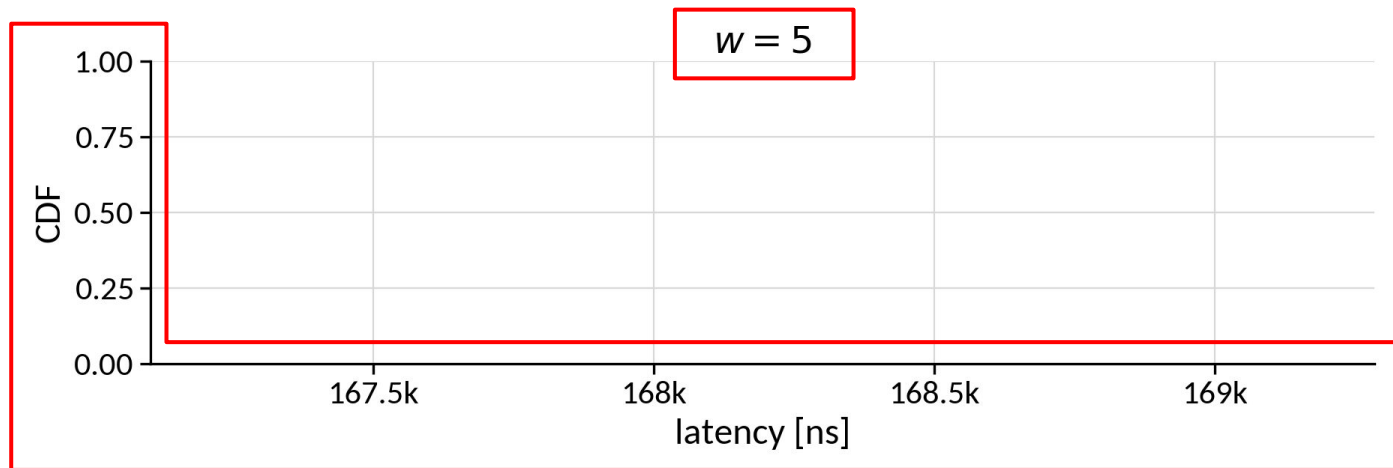
Recommendation: Pin packet processing to one single core host-side

Predicting the Worst Case

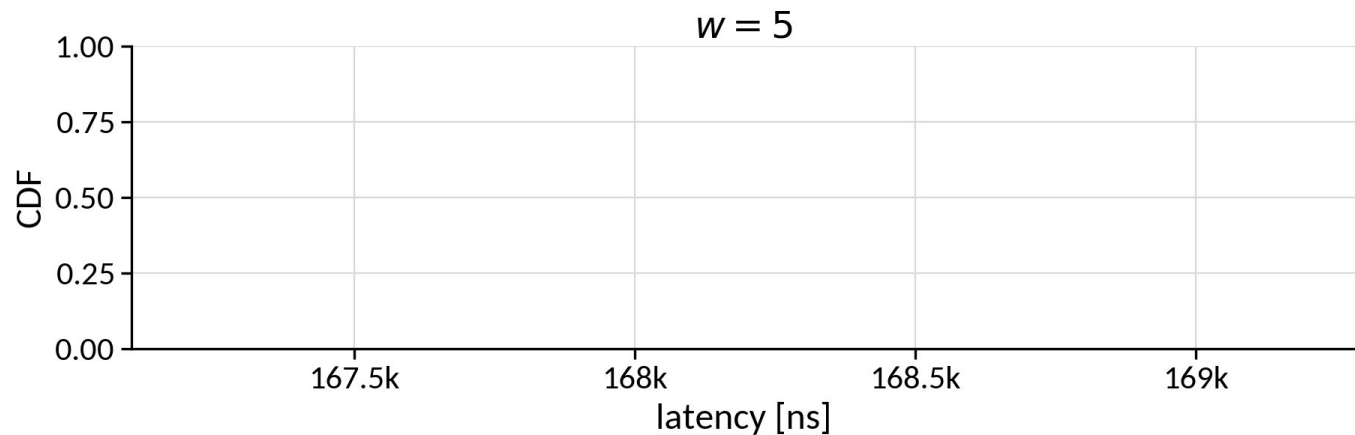


- **The scenario**
 - Five SV (Sampled Values) windows hit the DPDK stack at once
- **Cost of one extra window**
 - According to our latency decomposition derivation: only VM costs
- **Two models**
 - Constant
 - Constant + Send + IPC, with the last two as random variables

Prediction Error



Prediction Error



Δ [ns]

p_{50}

p_{99}

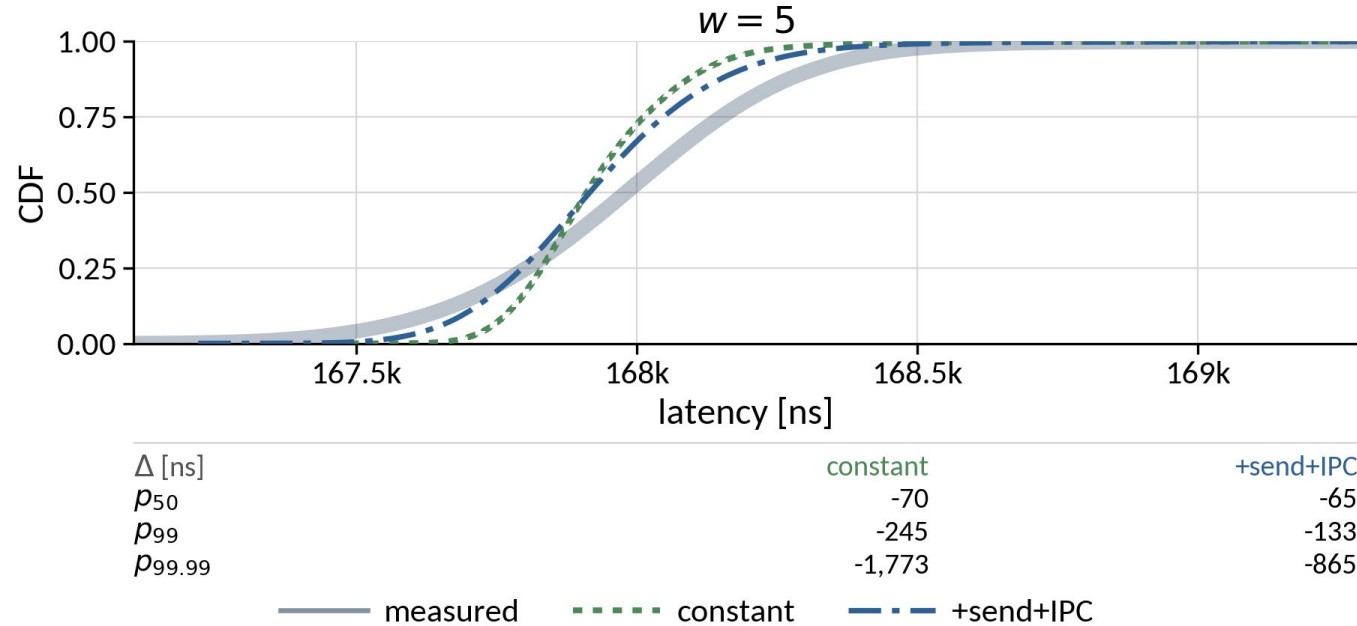
$p_{99.99}$

constant

+send+IPC

— measured - - - constant - . - +send+IPC

Prediction Error



Modelling the variability halves the tail error: 0.9 μ s vs 1.8 μ s at $p_{99.99}$

Conclusion

Can software meet the protection deadline of 5 ms, and for how many streams?

=> Yes, for up to 128 streams, for all stacks.

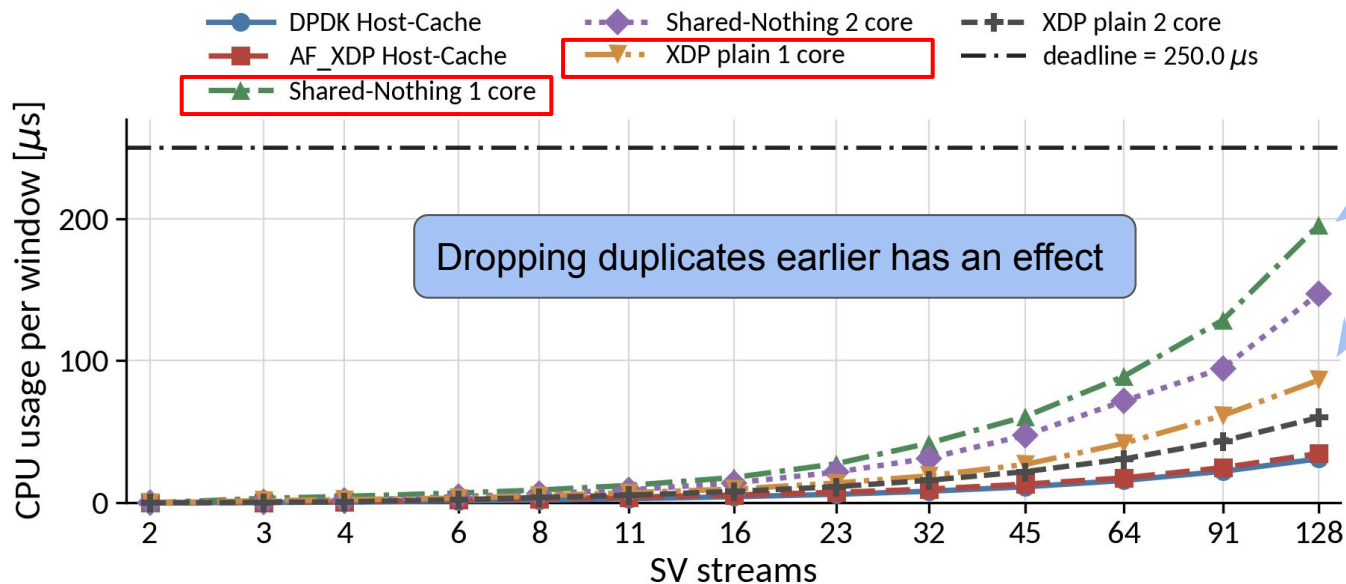
- **Contributions:**

- **Design and implement** the network stacks, leveraging kernel bypassing
- **Evaluate** the stacks from within the same framework
- **Model** capacities and the worst-case burst scenario for the DPDK Host-Cache variant

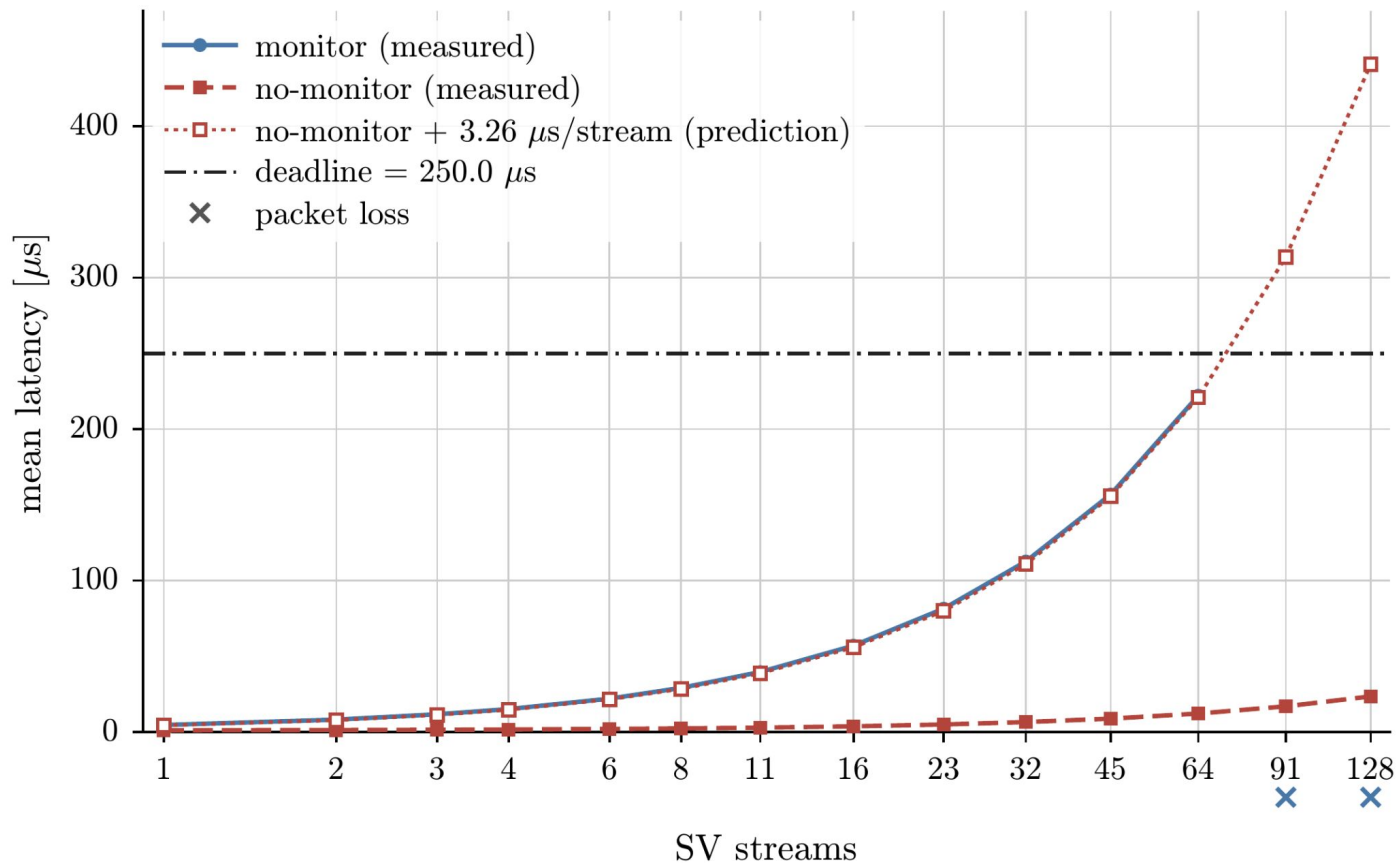
Stack	Shared-Nothing	AF_XDP Host-Cache	DPDK Host-Cache
Recommendation	Small and medium substations	Large substations, for ease of integration	Large substations, for determinism

Backup

Host-side Capacity

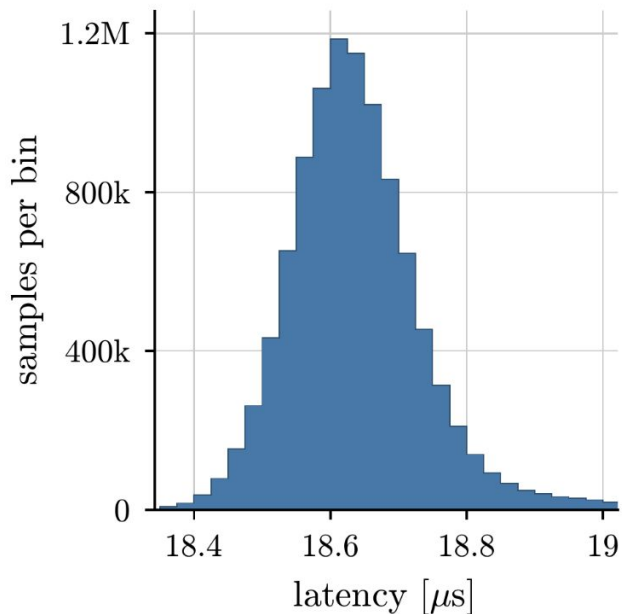


VM Capacity for the Host-Cache Variants



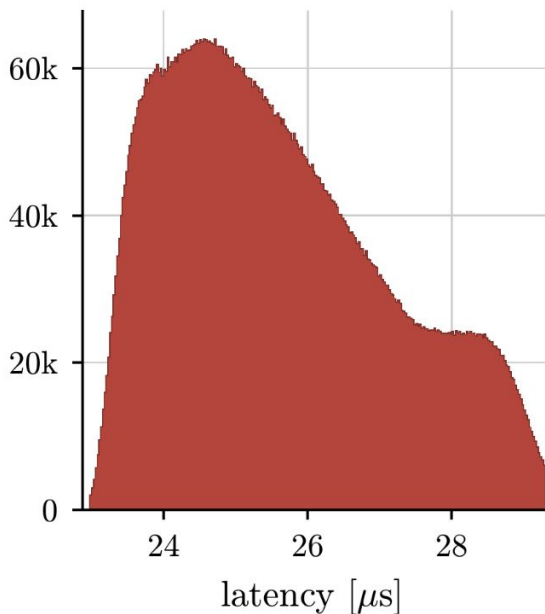
End-to-End Latency

DPDK Host-Cache



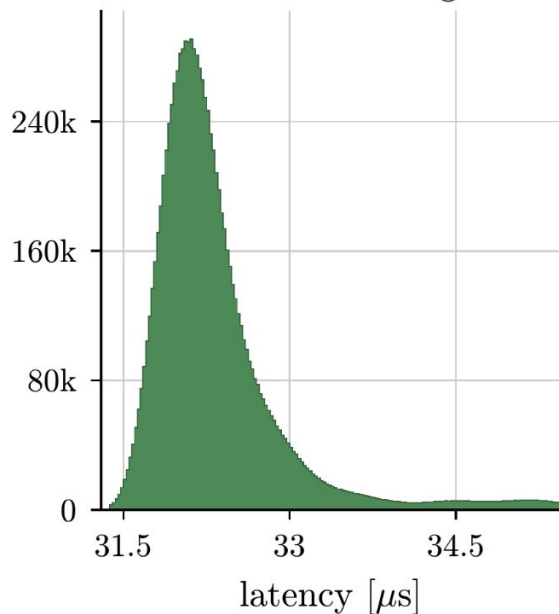
mean	18.6 μs
p_{50}	18.6 μs
p_{99}	19.0 μs
$p_{99.9}$	19.7 μs
$p_{99.99}$	21.2 μs
max	143.4 μs

AF_XDP Host-Cache



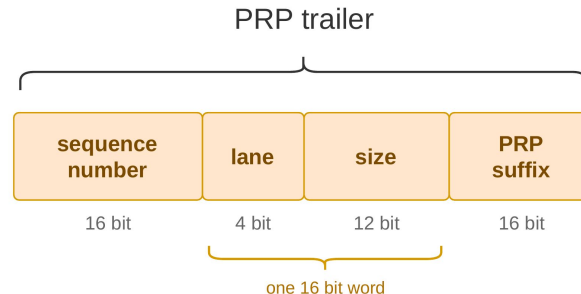
mean	25.7 μs
p_{50}	25.4 μs
p_{99}	29.3 μs
$p_{99.9}$	30.7 μs
$p_{99.99}$	33.3 μs
max	378.1 μs

Shared-Nothing



mean	32.4 μs
p_{50}	32.2 μs
p_{99}	35.5 μs
$p_{99.9}$	38.0 μs
$p_{99.99}$	45.0 μs
max	437.0 μs

- **Problem**
 - Receiver has to identify first arrival, discarding the duplicate
 - The source MAC and the RCT sequence number uniquely identify the frame
- **Our algorithm**
 - An epoch array and a valid array per network node
 - Sequence number = epoch (first 4 bits) + index (last 12 bits)
 - Accept if the entry is invalid or holds a different epoch



Sources

[1] S. Schönborn, R. Birke, D. Kozhaya, and T. Sivanthi. “Real-time performance of virtualised protection and control software.” In: 27th International Conference on Electricity Distribution (CIRED 2023). Vol. 2023. 2023, pp. 1817–1821. doi: 10.1049/icp.2023.1028.

[2] ABB. Virtualization of protection and control – Evaluation and deployment considerations. 2023. URL:
https://library.e.abb.com/public/3f5ff4e1c22b49a28987ffcac74eae96/White%20paper%20N GA001642_RevA_EN_finalb.pdf

[3] Intel. Virtual Protection Relay White Paper. 2024. URL:
<https://builders.intel.com/docs/networkbuilders/virtual-protection-relay-white-paper-1709299398.pdf>